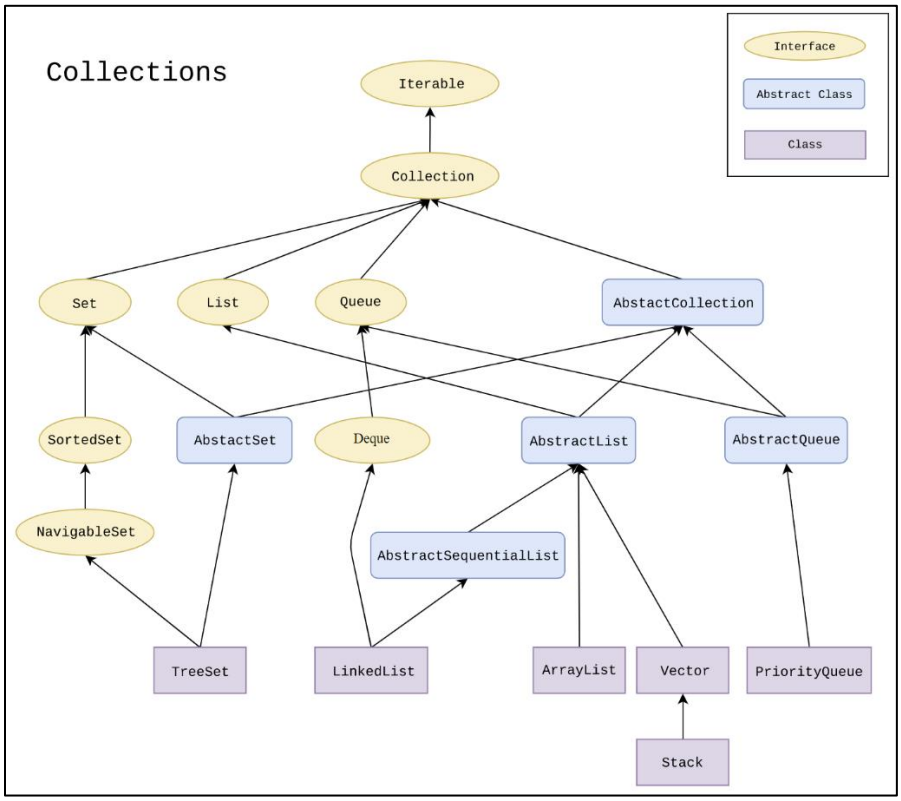
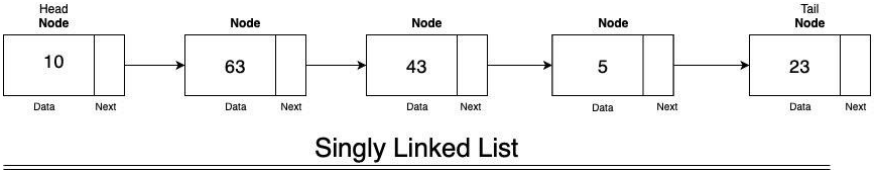
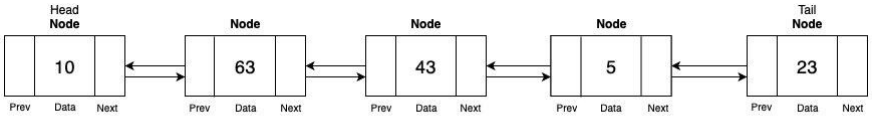
PKG 13: VECTOR, LISTs, and ITERATORs



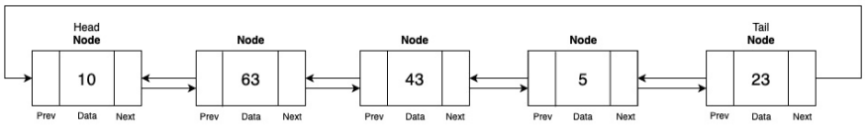
LISTs | Examples



Singly Linked List



Doubly Linked List



Circular Linked List

- Noncontiguous memory
- No pre-allocated memory
- Sorting, splicing, removing, unique
- Constant memory overhead

- Contiguous memory
- Random access
- Shifting
- Invalidation

- Insertion
- Deletion
- Caching

VECTOR

VECTOR

```
import java.util.Vector;

public class VectorDemo {

    public static void main(String[] args) {

        // 49+ methods => practice

        // The interesting

        Vector<String> vec = new Vector<String>();
        System.out.println("Size: " + vec.size());         // 0
        System.out.println("Capacity: " + vec.capacity()); // 10 initial capacity

        vec.ensureCapacity(20);
        System.out.println("Size: " + vec.size());         // 0
        System.out.println("Capacity: " + vec.capacity()); // 20

        vec.setSize(5);
        System.out.println("Size: " + vec.size());         // 5
        System.out.println("Capacity: " + vec.capacity()); // 20

        vec.setSize(20);
        System.out.println("Size: " + vec.size());         // 20
        System.out.println("Capacity: " + vec.capacity()); // 20

        vec.setSize(35);
        System.out.println("Size: " + vec.size());         // 35
        System.out.println("Capacity: " + vec.capacity()); // 40

        vec.setSize(45);
        System.out.println("Size: " + vec.size());         // 45
        System.out.println("Capacity: " + vec.capacity()); // 80

        vec.setSize(81);
        System.out.println("Size: " + vec.size());         // 81
        System.out.println("Capacity: " + vec.capacity()); // 160

        vec.trimToSize();
        System.out.println("Capacity: " + vec.capacity()); // 81


        // size:           returns the number of components in this vector
        // capacity:       returns the current capacity of this vector

        // setSize:        sets the size of this vector

        // ensureCapacity: increases the capacity of this vector, if necessary,
        //                 to ensure that it can hold at least the number of
        //                 components specified by the minimum capacity argument

        // trimToSize:     trims the capacity of this vector to be the vector's
        //                 current size

    }
}
```

ARRAYLIST

```java
import java.util.ArrayList;    import java.util.Arrays;
import java.util.Collections;  import java.util.Iterator;

public class ArrayListDemo {
    public static void main(String args[]) {
        ArrayList<String> myList = new ArrayList<>();

        myList.add("E1");
        myList.add("E2");
        myList.add(2, "E3");
        myList.add("E4");
        myList.add("E3");
        myList.add("E3");
        myList.add("E7");
        myList.add("E5");
        System.out.println(myList); // [E1, E2, E3, E4, E3, E3, E7, E5]

        System.out.println("");
        System.out.println(myList.isEmpty());         // false
        System.out.println(myList.size());            // 8
        System.out.println(myList.indexOf("E2"));     // 1
        System.out.println(myList.indexOf("E22"));    // -1
        System.out.println(myList.contains("E5"));    // true
        System.out.println(myList.get(0));            // E1

        System.out.println("");
        for (int i = 0; i < myList.size(); i++) {
            System.out.println(i + " ---> " + myList.get(i));
        }

        System.out.println("");
        for (String entry : myList) {
            System.out.println("Entry --> " + entry);
        }

        System.out.println("");
        for (Iterator<String> it = myList.iterator(); it.hasNext();) {
            System.out.println("Entry --> " + it.next());
        }

        System.out.println("");
        System.out.println(myList.size()); // 8
        myList.set(1, "NewE");
        System.out.println(myList.size()); // 8
        System.out.println(myList);        //[E1, NewE, E3, E4, E3, E3, E7, E5]
        myList.remove(1);
        System.out.println(myList.size()); // 7
        System.out.println(myList);        //[E1, E3, E4, E3, E3, E7, E5]

        System.out.println("");
        myList.remove(0);
        myList.remove("E3"); // remove the first occurrence of
        System.out.println(myList); // [E4, E3, E3, E7, E5]

        System.out.println("");
        Collections.sort(myList);
        System.out.println(myList); // [E3, E3, E4, E5, E7]

        System.out.println("");
        String[] newArray = myList.toArray(new String[myList.size()]);
        System.out.println(Arrays.toString(newArray)); // [E3, E3, E4, E5, E7]
```

```
0 ---> E1
1 ---> E2
2 ---> E3
3 ---> E4
4 ---> E3
5 ---> E3
6 ---> E7
7 ---> E5

Entry --> E1
Entry --> E2
Entry --> E3
Entry --> E4
Entry --> E3
Entry --> E3
Entry --> E7
Entry --> E5

Entry --> E1
Entry --> E2
Entry --> E3
Entry --> E4
Entry --> E3
Entry --> E3
Entry --> E7
Entry --> E5
```

LINKEDLIST

```
import java.util.Collections;
import java.util.LinkedList;
import java.util.ListIterator;

public class LinkedListDemo {

    public static void main(String args[]) {
        LinkedList myList = new LinkedList();

        myList.add("One");
        myList.add("Two");
        myList.add("Three");
        myList.add("Four");
        myList.add("Five");
        System.out.println(myList); // [One, Two, Three, Four, Five]

        ListIterator lit = myList.listIterator();
        while (lit.hasNext()) {
            System.out.println(lit.next());
        }

        System.out.println("");
        myList.addFirst("Zero");
        myList.addLast("Six");
        System.out.println(myList); // [Zero, One, Two, Three, Four, Five, Six]

        System.out.println("");
        System.out.println(myList.contains("Four")); // true
        System.out.println(myList.indexOf("One"));   // 1
        System.out.println(myList.get(2));           // Two
        System.out.println(myList.size());           // 7

        System.out.println("");
        lit = myList.listIterator(myList.size());
        while (lit.hasPrevious()) {
            System.out.println(lit.previous());
        }

        System.out.println("");
        myList.remove("Three");
        System.out.println(myList.size()); // 6
        myList.set(0, "NewZero");
        System.out.println(myList.size()); // 6
        myList.removeFirst();
        myList.removeLast();
        System.out.println(myList);        // [One, Two, Four, Five]

        System.out.println("");
        System.out.println(myList.size());
        myList.remove(1);
        System.out.println(myList.size());
        System.out.println(myList);         // [One, Four, Five]

        Collections.sort(myList);

        lit = myList.listIterator();
        while (lit.hasNext()) {
            System.out.println(lit.next());
        }
    }
}
```

```
One
Two
Three
Four
Five
```

```
Six
Five
Four
Three
Two
One
Zero
```

```
Five
Four
One
```

ITERATORS

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Iterator;
import java.util.ListIterator;

public class ListDemo {

    private static class Person implements Cloneable {

        private String name;

        private Person(String name) {
            this.name = name;
        }

        private Person(Person person) {
            this.name = person.getName();
        }

        private void setName(String name) {
            this.name = name;
        }

        private String getName() {
            return this.name;
        }

        @Override
        public String toString() {
            return this.name;
        }

        @Override
        protected Person clone() throws CloneNotSupportedException {
            Person clone = null;
            try {
                clone = (Person) super.clone();
            } catch (CloneNotSupportedException e) {
                throw new RuntimeException(e);
            }
            return clone;
        }
    }


    public static void main(String args[]) throws CloneNotSupportedException {

        ArrayList<String> myList = new ArrayList<>();
        ArrayList<String> yourList = new ArrayList<>();

        // add
        System.out.println("Add");
        myList.add("Mickey");
        myList.add("Minnie");
        myList.add("Goofy");
        myList.add("Dory");
        myList.add("Nemo");
        System.out.println(myList.size());  // 5
        System.out.println(myList); // [Mickey, Minnie, Goofy, Dory, Nemo]

```

```
        // "collapse"
        System.out.println("\nCollapse");
        myList.remove("Goofy");
        System.out.println(myList.size());  // 4
        System.out.println(myList); // [Mickey, Minnie, Dory, Nemo]

        // "make room"
        System.out.println("\nMake Room");
        myList.add(2, "Goofy");
        System.out.println(myList.size());  // 5
        System.out.println(myList); // [Mickey, Minnie, Goofy, Dory, Nemo]

        // "replace"
        System.out.println("\nReplace");
        myList.set(2, "Cinderella");
        System.out.println(myList.size());  // 5
        System.out.println(myList); // [Mickey, Minnie, Cinderella, Dory, Nemo]

        // "Shallow" Copy - immutable
        System.out.println("\nShallow Copy - immmutable");
        String tmpStr = "Mulan";
        myList.set(2, tmpStr);
        yourList = (ArrayList<String>) myList.clone();
        System.out.println(yourList);// [Mickey, Minnie, Mulan, Dory, Nemo]
        tmpStr = "Ariel"; // never used
        System.out.println(myList);  // [Mickey, Minnie, Mulan, Dory, Nemo]
        System.out.println(yourList);// [Mickey, Minnie, Mulan, Dory, Nemo]
        myList.set(2, "Cinderella");
        System.out.println(myList);  // [Mickey, Minnie, Cinderella, Dory, Nemo]
        System.out.println(yourList);// [Mickey, Minnie, Mulan, Dory, Nemo]

        // "Shallow" Copy - mutable
        System.out.println("\nShallow Copy - mutable");
        Person person1 = new Person("Messi");
        Person person2 = new Person("Neymar");
        ArrayList<Person> myList2 = new ArrayList<>();
        ArrayList<Person> yourList2 = new ArrayList<>();
        myList2.add(person1);
        myList2.add(person2);
        System.out.println("myList2:  \t" + myList2);     // [Messi, Neymar]
        yourList2 = (ArrayList<Person>) myList2.clone();
        System.out.println("yourList2:\t" + yourList2);   // [Messi, Neymar]
        person1.setName("Ronaldo");
        System.out.println("myList2:  \t" + myList2);     // [Ronaldo, Neymar]
        System.out.println("yourList2:\t" + yourList2);   // [Ronaldo, Neymar]

        // "Deep" Copy - mutable
        System.out.println("\nDeep Copy - mutable");
        ArrayList<Person> yourList3 = new ArrayList<>();
        Iterator<Person> iterator = myList2.iterator();
        while (iterator.hasNext()) {
            yourList3.add((Person) iterator.next().clone());
        }
        System.out.println("myList2:  \t" + myList2);    // [Ronaldo, Neymar]
        System.out.println("yourList3:\t" + yourList3);  // [Ronaldo, Neymar]
        person1.setName("Coutinho");
        System.out.println("myList2:  \t" + myList2);    // [Coutinho, Neymar]
        System.out.println("yourList3:\t" + yourList3);  // [Ronaldo, Neymar]

        // Traverse 1-2-3, for loop, while loop, do-while loop
        System.out.println("\nTraverse 1-2-3");
        for (int i = 0; i < myList.size(); i++) {
            System.out.print(myList.get(i) + " ");
```

```
377         }
378
379         // Traverse 4, for each
380         System.out.println("\nTraverse 4");
381         for (String entry : myList) {
382             System.out.print(entry + " ");
383         }
384
385         // Traverse 5, Iterator
386         System.out.println("\nTraverse 5");
387         for (Iterator<String> it = myList.iterator(); it.hasNext();) {
388             System.out.print(it.next() + " ");
389         }
390
391         // Traverse 6, Iterator
392         System.out.println("\nTraverse 6");
393         for (ListIterator<String> it = myList.listIterator(); it.hasNext();) {
394             System.out.print(it.next() + " ");
395         }
396
397         // Traverse 7, Iterator
398         System.out.println("\nTraverse 7");
399         for (ListIterator<String> it = myList.listIterator(myList.size());
400                                                                 it.hasPrevious();)
401
402         {
403             System.out.print(it.previous() + " ");
404         }
405
406         // Traverse 8, stream()
407         System.out.println("\nTraverse 8");
408         myList.stream().forEach(entry -> System.out.print(entry + " "));
409
410         // Collections, sort
411         System.out.println("\n\nCollections.sort");
412         Collections.sort(myList);
413         System.out.println(myList);
414
415         // toArray()
416         System.out.println("\ntoArray()");
417         String[] newArray = myList.toArray(new String[myList.size()]);
418         System.out.println(Arrays.toString(newArray));
419     }
420 }
421
422 Traverse 1-2-3
423 Mickey Minnie Cinderella Dory Nemo
424 Traverse 4
425 Mickey Minnie Cinderella Dory Nemo
426 Traverse 5
427 Mickey Minnie Cinderella Dory Nemo
428 Traverse 6
429 Mickey Minnie Cinderella Dory Nemo
430 Traverse 7
431 Nemo Dory Cinderella Minnie Mickey
432 Traverse 8
433 Mickey Minnie Cinderella Dory Nemo
434
435 Collections.sort
436 [Cinderella, Dory, Mickey, Minnie, Nemo]
437
438 toArray()
439 [Cinderella, Dory, Mickey, Minnie, Nemo]
```

440 ITERATORs

```
441
442 import java.util.List;
443 import java.util.LinkedList;
444 import java.util.Iterator;
445 import java.util.ListIterator;
446 import java.util.Collections;
447 import java.util.Arrays;
448
449 public class ListDemo2 {
450
451     public static void main(String args[]) {
452
453         List<String> myList = new LinkedList<>();
454
455         // add
456         System.out.println("Adding...");
457         myList.add("Mickey");
458         myList.add("Minnie");
459         myList.add("Goofy");
460         myList.add("Dory");
461         myList.add("Nemo");
462         System.out.println(myList); // [Mickey, Minnie, Goofy, Dory, Nemo]
463
464         // Traverse 1-2-3, for loop, while loop, do-while loop
465         System.out.println("\nTraverse 1-2-3");
466         for (int i = 0; i < myList.size(); i++) {
467             System.out.print(myList.get(i) + " ");
468         } // Mickey Minnie Goofy Dory Nemo
469
470         // Traverse 4, for each
471         System.out.println("\nTraverse 4");
472         for (String entry : myList) {
473             System.out.print(entry + " ");
474         } // Mickey Minnie Goofy Dory Nemo
475
476         // Traverse 5, for each, fuctional operations
477         System.out.println("\nTraverse 5");
478         myList.forEach((entry) -> {
479             System.out.print(entry + " ");
480         }); // Mickey Minnie Goofy Dory Nemo
481
482         // Traverse 6, Iterator
483         System.out.println("\nTraverse 6");
484         for (Iterator<String> it = myList.iterator(); it.hasNext();) {
485             System.out.print(it.next() + " ");
486         } // Mickey Minnie Goofy Dory Nemo
487
488         // Traverse 7, Iterator
489         System.out.println("\nTraverse 7");
490         for (ListIterator<String> li = myList.listIterator(); li.hasNext();) {
491             System.out.print(li.next() + " ");
492         } // Mickey Minnie Goofy Dory Nemo
493
494         // Traverse 8, Iterator
495         System.out.println("\nTraverse 8");
496         for (ListIterator<String> li = myList.listIterator(myList.size());
497                                                         li.hasPrevious();) {
498             System.out.print(li.previous() + " ");
499         } // Nemo Dory Goofy Minnie Mickey
500
501         // Traverse 9, stream()
502         System.out.println("\nTraverse 9");
```

```
503         myList.stream().forEach(entry -> System.out.print(entry + " "));
504         // Mickey Minnie Goofy Dory Nemo
505
506         // Collections, sort
507         System.out.println("\n\nCollections.sort");
508         Collections.sort(myList);
509         System.out.println(myList);
510         // [Dory, Goofy, Mickey, Minnie, Nemo]
511
512         // toArray()
513         System.out.println("\ntoArray()");
514         String[] newArray = myList.toArray(new String[myList.size()]);
515         System.out.println(Arrays.toString(newArray));
516         // [Dory, Goofy, Mickey, Minnie, Nemo]
517
518         // ListItertor, MORE
519         System.out.println("\nMORE...");
520         System.out.println(myList); // Dory, Goofy, Mickey, Minnie, Nemo
521         ListIterator liIdx = myList.listIterator(2);
522
523         System.out.println(liIdx.nextIndex()); // 2
524         System.out.println(liIdx.next());      // Mickey
525         System.out.println("remove 1");
526         liIdx.remove();
527         System.out.println(myList); // Dory, Goofy, Minnie, Nemo
528
529         System.out.println(liIdx.hasNext());   // true
530         System.out.println(liIdx.nextIndex()); // 2
531         System.out.println(liIdx.next());      // Minnie
532         System.out.println("remove 2");
533         liIdx.remove();
534         System.out.println(myList); // Dory, Goofy, Nemo
535
536         System.out.println(liIdx.nextIndex()); // 2
537         System.out.println("Add Minnie2");
538         liIdx.add("Minnie2");
539         System.out.println(myList); // Dory, Goofy, Minnie2, Nemo
540         System.out.println(liIdx.nextIndex()); // 3
541         System.out.println(liIdx.next());      // Nemo
542         System.out.println("Replace Neo");
543         liIdx.set("NemoReplaced");
544         System.out.println(myList); // Dory, Goofy, Minnie2, NemoReplaced
545         System.out.println(liIdx.nextIndex()); // 4
546         System.out.println(liIdx.hasNext());   // false
547
548         // forEachRemaining
549         liIdx = myList.listIterator(0);
550         liIdx.forEachRemaining(entry -> System.out.print(entry + " "));
551         // Dory Goofy Minnie2 NemoReplaced
552
553         System.out.println("");
554         liIdx = myList.listIterator(0);
555         liIdx.forEachRemaining(entry ->
556                         System.out.print(entry.getClass().getSimpleName() + " "));
557         // String String String String
558
559         // previous
560         System.out.println(liIdx.hasPrevious());    // true
561         System.out.println(liIdx.previousIndex());  // 3
562         System.out.println(liIdx.previous());       // NemoReplaced
563     }
564 }
565
```

VECTOR

The Vector class implements a growable array of objects. Like an array, it contains components that can be accessed using an integer index. However, the size of a Vector can grow or shrink as needed to accommodate adding and removing items after the Vector has been created.

Each vector tries to optimize storage management by maintaining a capacity and a capacityIncrement. The capacity is always at least as large as the vector size; it is usually larger because as components are added to the vector, the vector's storage increases in chunks the size of capacityIncrement. An application can increase the capacity of a vector before inserting a large number of components; this reduces the amount of incremental reallocation.

The iterators returned by this class's iterator and listIterator methods are fail-fast: if the vector is structurally modified at any time after the iterator is created, in any way except through the iterator's own remove or add methods, the iterator will throw a ConcurrentModificationException. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future. The Enumerations returned by the elements method are not fail-fast; if the Vector is structurally modified at any time after the enumeration is created then the results of enumerating are undefined.
...

LIST

An ordered collection (also known as a sequence). The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index (position in the list), and search for elements in the list.

Unlike sets, lists typically allow duplicate elements. More formally, lists typically allow pairs of elements e1 and e2 such that e1.equals(e2), and they typically allow multiple null elements if they allow null elements at all. It is not inconceivable that someone might wish to implement a list that prohibits duplicates, by throwing runtime exceptions when the user attempts to insert them, but we expect this usage to be rare.

The List interface places additional stipulations, beyond those specified in the Collection interface, on the contracts of the iterator, add, remove, equals, and hashCode methods. Declarations for other inherited methods are also included here for convenience.

The List interface provides four methods for positional (indexed) access to list elements. Lists (like Java arrays) are zero based. Note that these operations may execute in time proportional to the index value for some implementations (the LinkedList class, for example). Thus, iterating over the elements in a list is typically preferable to indexing through it if the caller does not know the implementation.

The List interface provides a special iterator, called a ListIterator, that allows element insertion and replacement, and bidirectional access in addition to the normal operations that the Iterator interface provides. A method is provided to obtain a list iterator that starts at a specified position in the list.

The List interface provides two methods to search for a specified object. From a performance standpoint, these methods should be used with caution. In many implementations they will perform costly linear searches.

...